

Parte 4 — A União: o cérebro no ESP32-S3

Onde estamos na jornada. Na Parte 1 montamos o corpo do robô. Na Parte 3 construímos e treinamos o cérebro (a LLM) no computador. Agora vem o encontro: vamos **levar o cérebro para dentro do robô** — fazer o modelo rodar num segundo microcontrolador, o ESP32-S3, e conversar com o ESP32 do corpo.

Esta é a parte que transforma um "modelo que roda no PC" num **robô de verdade que fala sozinho**. É também a mais avançada do projeto, então vamos com calma, testando cada peça.

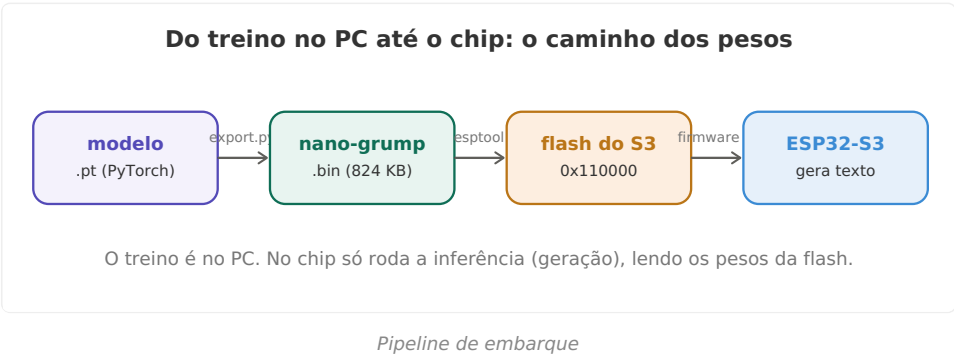
Nota sobre a versão do modelo. Para o embarque, o nano-grump foi turbinado para uma versão 2.0 mais capaz — ainda minúscula, mas com mais "musculatura" para falar melhor rodando sozinho no chip. As dimensões usadas nesta parte são: vocab_size=59, n_embd=64, block_size=64, n_layer=4, n_heads=4 (atenção de 4 cabeças), com ~211 mil parâmetros. A arquitetura é a mesma que você aprendeu na Parte 3 — só um pouco maior. Se você seguiu a Parte 3 com a versão de 42 mil parâmetros, o processo aqui é idêntico; só mudam os números do cabeçalho.

4.1 O problema: dois idiomas diferentes

Seu modelo foi treinado em **PyTorch**, que é Python — um ambiente pesado, impossível de rodar num microcontrolador. O ESP32-S3 roda **C puro**, lendo os dados direto da memória flash.

A analogia: o `modelo_treinado.pt` é uma receita escrita em português; o ESP32-S3 só lê receitas em japonês. Precisamos **traduzir a receita** — sem mudar nenhum ingrediente nem passo, só reescrever no idioma que o chip entende.

O caminho completo, do PC ao chip:



Repare num ponto fundamental: **o treino acontece no PC**. O chip só roda a **inferência** (a geração de texto). O modelo não "aprende" no robô — ele já vem treinado, e no chip apenas lê os pesos e gera frases.

Vamos dividir o embarque em três etapas:

1. **Exportar** — um script Python que traduz o `.pt` num arquivo `.bin`
2. **Gravar e ler** — colocar o `.bin` na flash e o firmware C lê os pesos
3. **Comunicar** — os dois ESP32 conversam (corpo → cérebro → frase)

4.2 Etapa 1 — Exportar os pesos (.pt → .bin)

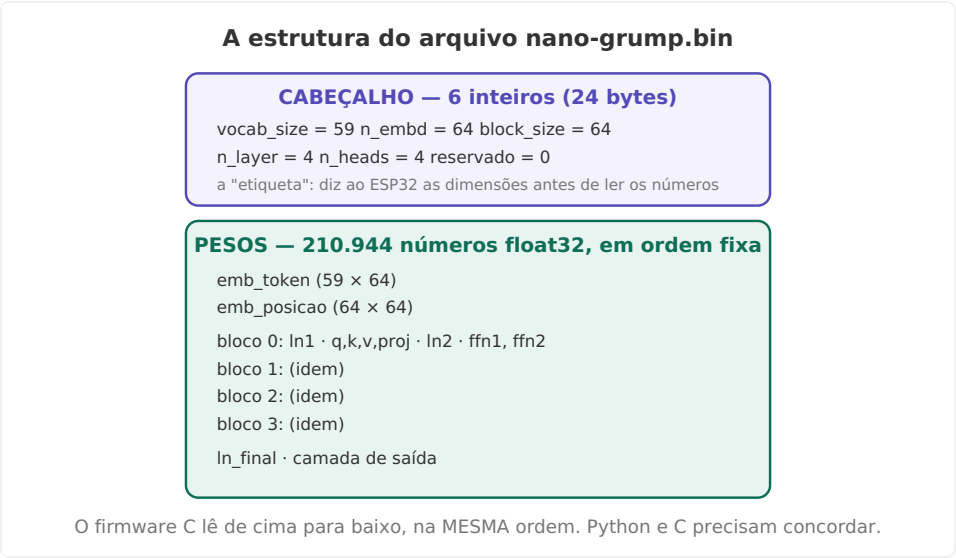
A ideia em uma frase: o script de exportação pega os ~211 mil números do modelo treinado, converte todos para um formato universal (float32), e salva num arquivo binário com uma "etiqueta" no começo.

O que é um arquivo binário? É a diferença entre um `.txt` (que você abre e lê) e um arquivo que só o computador entende — uma sequência de bytes puros. O `.bin` é desse segundo tipo: números empacotados de forma que o C leia rapidamente.

O que são os "pesos"? O modelo treinado é, no fundo, uma **coleção de números** — os ~211 mil valores que o treino ajustou. São eles que dão personalidade ao robô. Exportar é só empacotar esses números num formato que o C leia.

A estrutura do arquivo .bin

O arquivo tem duas partes, nesta ordem:



Estrutura do arquivo .bin

A **ordem dos pesos tem que ser idêntica** nos dois lados: o `export.py` escreve numa ordem, e o firmware C lê exatamente na mesma. É como combinar previamente em que gaveta fica cada coisa.

O código do export.py

```
"""
export.py – traduz o modelo_treinado.pt para nano-grump.bin
Uso: uv run export.py
Saída: nano-grump.bin (pronto para gravar no ESP32-S3)
"""
import struct
from pathlib import Path
import torch
from model import MiniGPT, vocab_size, n_embd, block_size, n_layer, n_heads

pasta = Path(__file__).parent
modelo = MiniGPT()
modelo.load_state_dict(torch.load(pasta / "modelo_treinado.pt", map_location="cpu"))
modelo.eval()

# Converte um tensor PyTorch em bytes float32 (4 bytes por número)
def para_bytes(tensor):
    return tensor.detach().cpu().numpy().astype("f").tobytes()

with open(pasta / "nano-grump.bin", "wb") as f: # "wb" = escrita binária
    # BLOCO 1: cabeçalho – 6 inteiros (a "etiqueta")
    f.write(struct.pack("6i", vocab_size, n_embd, block_size,
                        n_layer, n_heads, 0))

    # BLOCO 2: pesos, em ORDEM FIXA
    f.write(para_bytes(modelo.emb_token.weight))
    f.write(para_bytes(modelo.emb_posicao.weight))
    for bloco in modelo.blocos:
        f.write(para_bytes(bloco.ln1.weight)); f.write(para_bytes(bloco.ln1.bias))
        f.write(para_bytes(bloco.atencao.query.weight))
        f.write(para_bytes(bloco.atencao.chave.weight))
        f.write(para_bytes(bloco.atencao.value.weight))
        f.write(para_bytes(bloco.atencao.proj.weight))
        f.write(para_bytes(bloco.atencao.proj.bias))
        f.write(para_bytes(bloco.ln2.weight)); f.write(para_bytes(bloco.ln2.bias))
        f.write(para_bytes(bloco.ffn.rede[0].weight))
        f.write(para_bytes(bloco.ffn.rede[0].bias))
        f.write(para_bytes(bloco.ffn.rede[2].weight))
        f.write(para_bytes(bloco.ffn.rede[2].bias))
    f.write(para_bytes(modelo.ln_final.weight)); f.write(para_bytes(modelo.ln_final.bias))
    f.write(para_bytes(modelo.saida.weight))

print("nano-grump.bin gerado.")
```

Traduzindo os pontos novos:

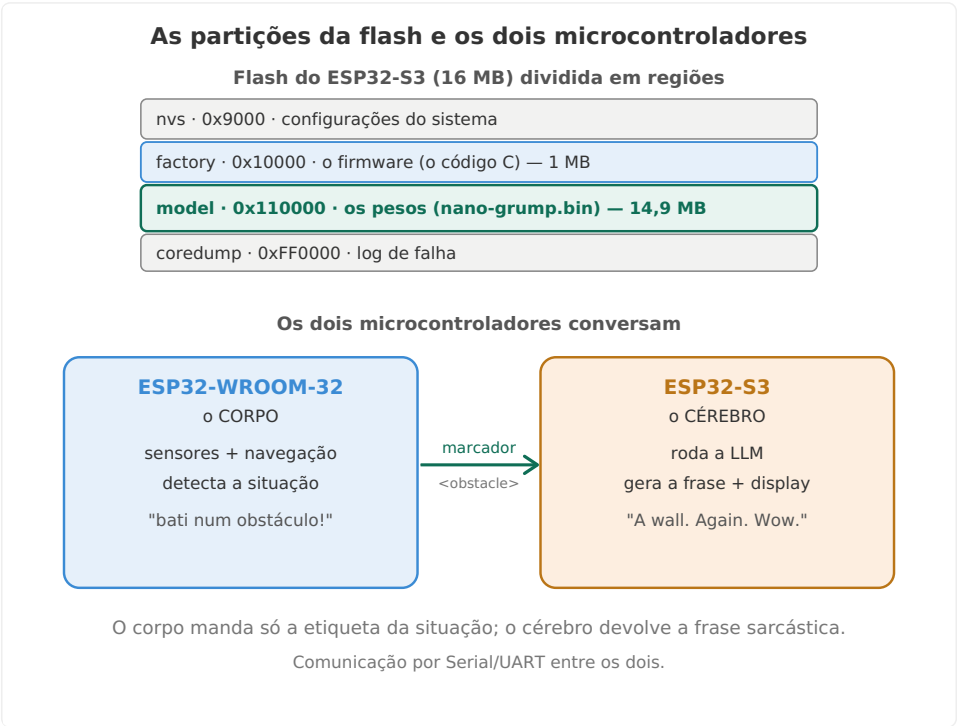
- `struct.pack("6i", ...)` empacota 6 inteiros em bytes — é o cabeçalho.
- `.astype("f")` converte cada número para **float32** (4 bytes), o formato que o C lê nativamente. É como trocar "xícaras" por "mililitros": mesmo valor, unidade universal.
- `"wb"` abre o arquivo em modo de **escrita binária**.

Rode com `uv run export.py`. Você deve ver a criação do `nano-grump.bin` de **~824 KB** (≈211 mil números × 4 bytes).

4.3 Etapa 2 — Gravar na flash e ler o cabeçalho

As partições da flash

A flash do ESP32-S3 (16 MB) é dividida em **regiões** (partições), como um HD. Um arquivo `partitions.csv` define essas regiões. A que nos interessa é a `model`, no endereço `0x110000`, com 14,9 MB — onde o `nano-grump.bin` (824 KB) cabe folgado.



Partições da flash e comunicação

Conteúdo do `partitions.csv`:

#	Name,	Type,	SubType,	Offset,	Size
nvs,	data,	nvs,		0x9000,	0x5000
factory,	app,	factory,		0x10000,	0x100000
model,	data,	0x40,		0x110000,	0xEF0000
coredump,	data,	coredump,		0xFF0000,	0x10000

Gravando o `.bin` com `esptool`

Conecte o ESP32-S3, segure o botão **BOOT**, e rode (troque `COM12` pela sua porta):

```
python -m esptool --chip esp32s3 --port COM12 write-flash 0x110000 nano-grump.bin
```

Você deve ver ao final: **Hash of data verified** — os bytes chegaram íntegros no endereço `0x110000`. Fazemos isso **antes** de qualquer firmware: se algo der errado, sabemos que foi na gravação dos dados, não no código.

Configurando o Arduino IDE

Para o firmware enxergar a partição `model`, o Arduino IDE precisa usar o `partitions.csv` customizado:

1. Coloque o `firmware.ino` e o `partitions.csv` **na mesma pasta**, e essa pasta deve ter o **mesmo nome do** `.ino` (ex: `firmware/firmware.ino`).
2. Em Ferramentas → Partition Scheme, selecione **Custom** (essa opção só aparece quando há um `partitions.csv` na pasta do

sketch).

3. Confirme também: Placa = **ESP32S3 Dev Module**, PSRAM = **OPI PSRAM**, Flash Size = **16MB**.

Pegadinha comum: o `.ino` precisa estar direto na pasta de mesmo nome (`firmware/firmware.ino`), não numa subpasta aninhada. Se o upload continuar indo para `0x10000` com esquema padrão, é sinal de que o Arduino IDE não achou o `partitions.csv`. E note: o **firmware** sempre vai para `0x10000` (partição `factory`); os **pesos** ficam em `0x110000` (partição `model`). Os dois coexistem.

Firmware mínimo: ler o cabeçalho (teste da peça isolada)

Antes da inferência, um firmware que só lê o cabeçalho e imprime os 6 números — para confirmar que a base está certa (a regra de ouro: testar uma peça de cada vez).

```
#include "esp_partition.h"

struct __attribute__((packed)) Cabecalho {
    int vocab_size, n_embd, block_size, n_layer, n_heads, reservado;
};

void setup() {
    Serial.begin(115200); delay(1500);

    // Encontrar a partição "model"
    const esp_partition_t *part = esp_partition_find_first(
        ESP_PARTITION_TYPE_DATA, (esp_partition_subtype_t)0x40, "model");
    if (!part) { Serial.println("ERRO: particao 'model' nao encontrada."); return; }
    Serial.printf("Particao: %.1f MB @ 0x%x\n", part->size/1048576.0, part->address);

    // Mapear na memória e ler os primeiros 24 bytes como Cabecalho
    const void *base; esp_partition_mmap_handle_t h;
    esp_partition_mmap(part, 0, part->size, ESP_PARTITION_MMAP_DATA, &base, &h);
    const Cabecalho *cab = (const Cabecalho *)base;

    Serial.printf("vocab_size=%d n_embd=%d block_size=%d n_layer=%d n_heads=%d\n",
        cab->vocab_size, cab->n_embd, cab->block_size, cab->n_layer, cab->n_heads);
}

void loop() {}
```

Grave e abra o Monitor Serial (115200 baud). Você deve ver `vocab_size=59 n_embd=64 block_size=64 n_layer=4 n_heads=4`. Se bater, a ponte Python → C está funcionando.

4.4 Etapa 3 — A inferência em C

Aqui o modelo ganha vida no chip. A boa notícia: **a matemática é idêntica** à que você aprendeu na Parte 3. O que muda é o idioma — em vez de o PyTorch cuidar de tudo, você diz explicitamente onde cada número está na memória.

Os três tipos de memória do ESP32-S3

Pense em três mesas de trabalho de tamanhos diferentes:

- **Flash** (16 MB) — mesa gigante, mas **só leitura**. Aqui ficam os pesos (824 KB).
- **PSRAM** (8 MB) — mesa grande, leitura e escrita. Aqui fica o "cache" da sequência.
- **SRAM** (512 KB) — mesa pequena e rápida. Aqui ficam os vetores de cálculo.

Os pesos nunca mudam → ficam na flash (lidos direto, sem copiar). Os cálculos mudam a cada caractere → ficam na SRAM/PSRAM.

As funções matemáticas (as mesmas peças, em C)

Cada operação da Parte 3 vira uma função em C:

```
// matmul: multiplica matriz por vetor – é o que o Linear() do PyTorch faz
static void matmul(float *out, const float *x,
    const float *A, int linhas, int colunas) {
    for (int i = 0; i < linhas; i++) {
        float acc = 0.0f;
```

```

    for (int j = 0; j < colunas; j++)
        acc += A[i * colunas + j] * x[j];
    out[i] = acc;
}
}

// softmax: converte notas em probabilidades que somam 1 (demo da atenção)
static void softmax(float *x, int n) {
    float mx = x[0];
    for (int i = 1; i < n; i++) if (x[i] > mx) mx = x[i];
    float soma = 0.0f;
    for (int i = 0; i < n; i++) { x[i] = expf(x[i] - mx); soma += x[i]; }
    for (int i = 0; i < n; i++) x[i] /= soma;
}

// relu: zera os negativos (a ativação da FFN)
static void relu(float *x, int n) {
    for (int i = 0; i < n; i++) if (x[i] < 0.0f) x[i] = 0.0f;
}

```

O forward pass em C (a tradução direta do model.py)

O coração da inferência é o mesmo fluxo da Parte 3: embedding → blocos (atenção + FFN com residual) → LayerNorm final → camada de saída. Cada token entra, e sai uma lista de notas (logits) para os 59 caracteres.

```

static void forward(int token, int pos) {
    // 1. EMBEDDING (caractere + posição) – Parte 3, seções 3.5–3.6
    const float *te = w.emb_token + token * N_EMBD;
    const float *pe = w.emb_pos + pos * N_EMBD;
    for (int i = 0; i < N_EMBD; i++) x[i] = te[i] + pe[i];

    // 2. BLOCOS (atenção multi-cabeça + FFN, com residual) – Parte 3, 3.7–3.11
    for (int l = 0; l < N_LAYER; l++) {
        // ... LayerNorm -> Q,K,V -> atenção por cabeça -> soma ponderada
        // ... + residual -> LayerNorm -> FFN (expandir, ReLU, contrair) -> residual
        // (a mesma sequência do bloco que você montou na Parte 3)
    }

    // 3. LAYERNORM FINAL + CAMADA DE SAÍDA -> logits (59 notas)
    rmsnorm(xb, x, w.ln_final_w, N_EMBD);
    matmul(logits, xb, w.saida_w, VOCAB_SIZE, N_EMBD);
}

```

O firmware completo (com a atenção multi-cabeça detalhada, o cache KV que guarda as keys e values das posições anteriores, e o mapeamento dos ponteiros para a flash) é longo. O ponto para você levar: **cada função é uma peça que você já entende da Parte 3, reescrita em C**. A atenção usa `matmul + softmax`, a FFN usa `matmul + relu`, e o cache KV é a forma prática de implementar a máscara causal (cada token só olha as posições já calculadas).

A geração (top-k + temperatura, como o generate.py)

O loop de geração é o mesmo ciclo autorregressivo da Parte 3: prevê → sorteia → anexa → repete. No chip, aplicamos temperatura e top-k antes do sorteio:

```

// Aplicar temperatura, manter só os TOP_K maiores logits, softmax, sortear
for (int v = 0; v < VOCAB_SIZE; v++) logits[v] /= TEMPERATURA;
// ... (zera todos menos os K maiores) ...
softmax(logits, VOCAB_SIZE);
int next_token = amostrar(logits, VOCAB_SIZE); // sorteio ponderado
if (next_token == TOKEN_NEWLINE) break;       // fim da frase

```

Os parâmetros ficam no topo do firmware, fáceis de ajustar: `TEMPERATURA = 0.75`, `TOP_K = 4` (os mesmos valores escolhidos no PC).

O vocabulário no firmware

O firmware precisa de uma cópia do vocabulário (os 59 caracteres) na mesma ordem do `vocab.json`, para traduzir números de volta em letras:

```
const char VOCAB[59] = {
```

```
{
    '\n', ' ', '\', ',', '-', '.', ':', '<', '>', '?',
    'A', 'B', 'C', 'D', 'E', 'F', 'G', 'H', 'I', 'J',
    'L', 'M', 'N', 'O', 'P', 'R', 'S', 'T', 'U', 'V',
    'W', 'Y', '_', 'a', 'b', 'c', 'd', 'e', 'f', 'g',
    'h', 'i', 'j', 'k', 'l', 'm', 'n', 'o', 'p', 'q',
    'r', 's', 't', 'u', 'v', 'w', 'x', 'y', 'z'
};
```

O robô usa um display OLED **SH1106 128×64** (SPI) para mostrar as frases. A biblioteca é a **U8g2**. Pinos confirmados no hardware: CLK=12, MOSI=11, CS=8, DC=9, RES=10.

```
#include <U8g2lib.h>

U8G2_SH1106_128X64_NONAME_F_4W_HW_SPI display(U8G2_R0, /*cs=*/8, /*dc=*/9, /*reset=*/10);

void setup() {
  display.begin();
  display.clearBuffer();
  display.setFont(u8g2_font_6x10_tf);
  display.drawStr(0, 12, "nano-grump v2");
  display.sendBuffer();
}
```

Conforme o modelo gera cada caractere, o firmware acumula a frase e a envia ao display, quebrando em linhas. Assim a reclamação sarcástica aparece na telinha do robô.

O sistema final tem **dois microcontroladores** com papéis distintos:

- **ESP32-WROOM-32 (o corpo)** — cuida dos sensores e da navegação (a Parte 1). Ele sabe quando bateu num obstáculo, quando ficou preso, etc.
- **ESP32-S3 (o cérebro)** — roda a LLM e o display. Ele sabe gerar as frases.

O corpo não precisa saber gerar frases; ele só manda a **etiqueta da situação** (o marcador) para o cérebro. O cérebro recebe, gera a frase e mostra no display:



No firmware do cérebro, o `loop()` fica escutando a Serial. Quando chega um marcador (começando com `<`), ele dispara a geração:

```
void loop() {
  while (Serial.available()) {
    char c = Serial.read();
    if (c == '\n' || c == '\r') {
      if (input_len > 0) {
        input_buf[input_len] = '\0';
        if (input_buf[0] == '<') {          // é um marcador?
          char prompt[34];
          snprintf(prompt, sizeof(prompt), "%s ", input_buf);
          gerar(prompt);                  // gera e exibe a frase
        }
        input_len = 0;
      }
    } else if (input_len < 31) {
      input_buf[input_len++] = c;
    }
  }
}
```

```
}
```

Para testar sem o robô inteiro montado, basta abrir o Monitor Serial do ESP32-S3 e digitar um marcador (`<obstacle>` , `<stuck>` , etc.). A frase aparece na Serial e no display — exatamente como apareceria vinda do corpo.

A ligação física entre os dois: conecta-se o pino TX do WROOM-32 ao RX do S3 (e os GNDs unidos). O corpo envia o texto do marcador; o cérebro escuta. É a mesma comunicação Serial que você usa entre o PC e a placa, só que entre duas placas.

Encerramento da Parte 4

Você levou o cérebro do PC para o hardware:

- ✔ Exportou os pesos do PyTorch para um `.bin` legível pelo C
- ✔ Gravou o `.bin` na flash do ESP32-S3 (partição `model` , `0x110000`)
- ✔ Leu o cabeçalho no firmware (a ponte Python → C confirmada)
- ✔ Entendeu a inferência em C (as mesmas peças da Parte 3, traduzidas)
- ✔ Integrou o display OLED
- ✔ Montou a comunicação entre corpo (WROOM-32) e cérebro (ESP32-S3)

O resultado é um robô autônomo com personalidade: ele explora, detecta obstáculos, e **reclama sarcasticamente** na telinha — com um modelo de linguagem que você construiu, treinou e embarcou, entendendo cada peça do caminho. 🤖

Nota de honestidade técnica. O firmware de inferência é a parte mais avançada e costuma exigir ajustes finos (alinhamento exato dos pesos, detalhes do LayerNorm, desempenho). Se algo não gerar texto coerente de primeira, é normal — depura-se uma peça de cada vez, começando pelo cabeçalho (que já validamos) e seguindo função por função. A jornada de entender cada etapa é o que torna esse ajuste possível.